

❖ · Neural Networks and Inductive Biases

1 The problem: modeling unknown functions

Scientific data is full of relationships we can measure but not write down. A telescope records stellar spectra; the mapping from spectrum to stellar composition is complex. A detector measures particle interactions; extracting physical parameters requires inverting a complicated function. We have (x, y) pairs and want to learn $y = f(x)$, but f has no closed form.

Linear models. The simplest approach: assume f is linear.

$$f(x) = w^\top x + b \tag{1.1}$$

Find w and b by minimizing squared error. This has a closed-form solution, is fast, and gives interpretable parameters. But linear models can only capture linear relationships.

Polynomial features. To capture curvature, transform the input: $\phi(x) = [1, x, x^2, x^3, \dots]$. Then fit a linear model in the transformed space:

$$f(x) = w^\top \phi(x) \tag{1.2}$$

The model is linear in parameters w , nonlinear in input x . This is *feature engineering*: choose transformations that expose the structure you care about.

The limitation. Feature engineering works when you know what transformations matter. Polynomials capture smooth curves. Fourier features capture periodicity. But in 100 dimensions, all quadratic interactions give 5,050 features. All cubics: over 170,000. Most are irrelevant, and you don't know which ones matter in advance. Hand-crafting features becomes impossible.

Neural networks sidestep this by learning features from data.

2 Neural networks: composing simple functions

A neural network builds complex functions by composing simple ones. Apply a linear transformation, then a simple nonlinearity, then another linear transformation, and so on:

$$h_1 = \sigma(W_1 x + b_1) \tag{2.1}$$

$$h_2 = \sigma(W_2 h_1 + b_2) \tag{2.2}$$

$$y = W_3 h_2 + b_3 \tag{2.3}$$

Each layer applies weights W_ℓ , biases b_ℓ , and a nonlinear activation σ . This is a *multilayer perceptron* (MLP).

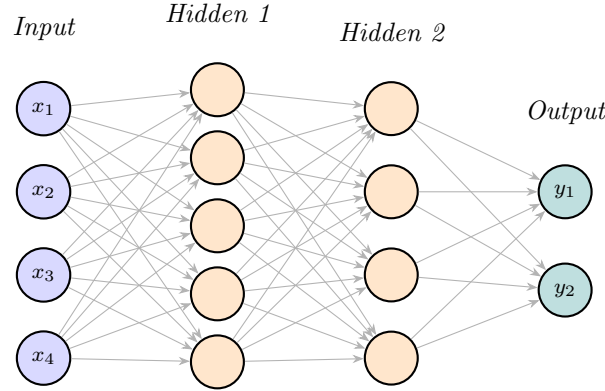


Figure 1. A multilayer perceptron with two hidden layers. Each layer applies a linear transformation followed by a nonlinearity. All neurons in adjacent layers are connected (“fully connected”).

Why nonlinearity matters. Without σ , the network collapses to a single linear map:

$$W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2) = W'x + b' \quad (2.4)$$

No matter how many layers, the result is linear. The nonlinearity is what gives depth its power.

What can nonlinearity do? Consider the XOR problem: two classes arranged so no single line can separate them (Figure 2). A linear model fails. But a network with one hidden layer can create a piecewise-linear decision boundary that correctly separates the classes. Adding more layers and neurons produces smoother, more complex boundaries.

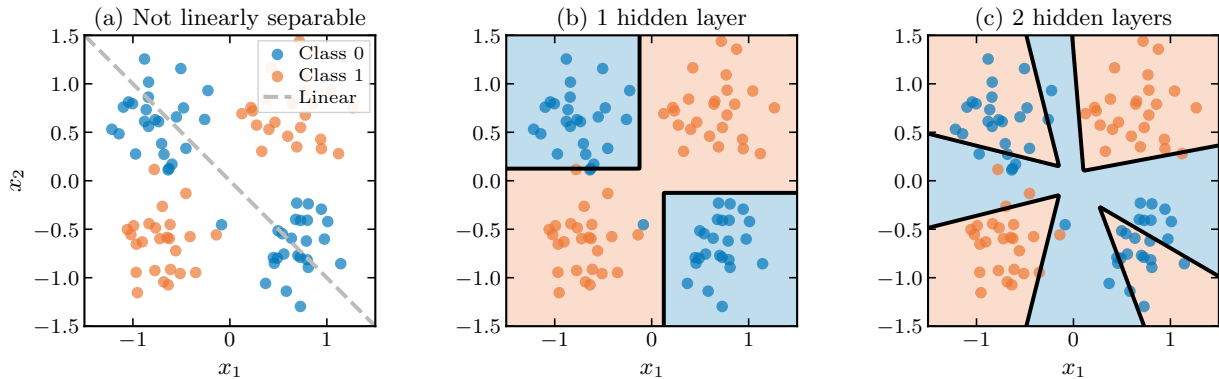


Figure 2. Nonlinearity enables complex decision boundaries. (a) An XOR-like classification problem: no straight line separates the classes. (b) A single hidden layer creates a piecewise-linear boundary. (c) A deeper network produces a smooth boundary that perfectly separates the data.

Common activations. The most common is ReLU:

$$\sigma(z) = \max(0, z) \quad (2.5)$$

Piecewise linear: identity for positive inputs, zero otherwise. Simple, fast, and effective. Variants (Leaky ReLU, GELU, SiLU) address edge cases but share the same core idea.

Universal approximation. With enough neurons, an MLP can approximate any continuous function. This is reassuring but not practically useful—it says nothing about how many parameters we need or whether we can find them by training.

3 Training deep networks: residual connections

Deeper networks can represent more complex functions, but they’re harder to train. Gradients must flow backward through many layers; they can vanish (shrink to zero) or explode (grow unboundedly).

Residual connections. Instead of learning $h' = F(h)$, learn the *residual* $h' = F(h) + h$. The network learns what to *add* to the input rather than computing the output from scratch.

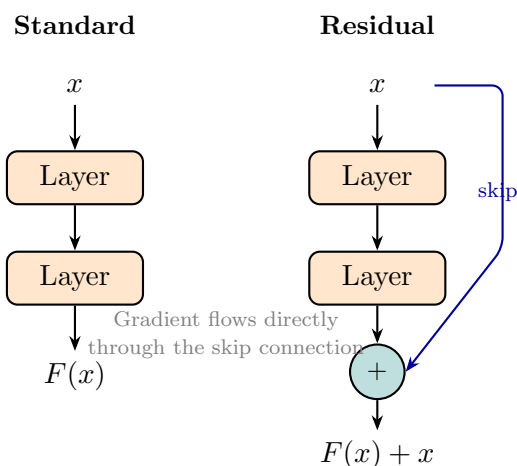


Figure 3. Residual (skip) connections. Left: a standard network computes $F(x)$. Right: a residual block computes $F(x) + x$. The skip connection provides a direct path for gradients, enabling training of very deep networks.

Why does this help? Gradients can flow directly through the skip connection, bypassing the layers entirely. Even if F contributes small gradients, the identity path ensures signal reaches early layers. This enables training networks with hundreds or thousands of layers.

Residual connections are now standard in almost all deep architectures: CNNs (ResNet), Transformers, and GNNs all use them.

4 Representation learning

The intermediate layers compute *representations*—transformed versions of the input that make the task easier.

The old way. Traditional ML: raw data $\rightarrow$ hand-crafted features $\rightarrow$ simple model $\rightarrow$ output. A domain expert designs features (bond lengths for molecules, color histograms for images). The model learns to map features to outputs.

The neural network way. Raw data $\rightarrow$ learned layers $\rightarrow$ learned representation $\rightarrow$ output. The network discovers useful features during training. For image classification, early layers might learn edge detectors, middle layers might learn textures and shapes, late layers might learn object parts.

The representation h_ℓ at layer ℓ is whatever the network found useful. If it learned well, h_ℓ exposes structure that makes prediction easy.

Transfer learning. Representations often transfer. A network trained on millions of natural images learns generic visual features—edges, textures, shapes—useful even for scientific images it never saw. Initialize with pretrained weights, fine-tune on your task. This is valuable when labeled scientific data is scarce.

5 The trouble with fully connected networks

Fully connected networks treat every input dimension as unrelated. For an image, pixel $(0,0)$ has no special relationship to pixel $(0,1)$, even though they're neighbors. The network must learn from scratch that:

- Nearby pixels are correlated
- Patterns can appear anywhere in the image
- A galaxy in the corner is the same galaxy as one in the center

With enough data, a fully connected network can learn this. But it's inefficient—we're asking the network to rediscover structure we already know. Scientific datasets are often small. Sample efficiency matters.

Better to build known structure into the architecture.

6 CNNs: weight sharing from translation symmetry

Images have two properties that fully connected networks ignore:

Translation invariance: An edge is an edge, regardless of where it appears. A galaxy in the corner should be classified the same as one in the center.

Locality: To understand a pixel, look at its neighbors. Distant pixels are (initially) irrelevant. Global structure emerges from composing local features.

These aren't assumptions we impose arbitrarily—they reflect the physics of spatial data. The inductive bias: patterns are local and can appear anywhere.

Deriving convolutions. Start with a fully connected layer:

$$h_{ij} = \sum_{k,l} W_{ijkl} x_{kl} \quad (6.1)$$

Weights W_{ijkl} connect input position (k,l) to output position (i,j) . For a megapixel image, this requires 10^{12} parameters.

Impose translation invariance: The same pattern should be detected identically everywhere. Weights cannot depend on absolute position—only on the offset. Let $a = k - i$, $b = l - j$:

$$W_{ijkl} = W_{ab} \quad \Rightarrow \quad h_{ij} = \sum_{a,b} W_{ab} x_{i+a,j+b} \quad (6.2)$$

This is a *convolution*. The same small set of weights (a *kernel*) slides across the image, computing the same operation at every position.

Impose locality: Only look at nearby pixels. Restrict to a small window:

$$h_{ij} = \sum_{|a| \leq k} \sum_{|b| \leq k} W_{ab} x_{i+a,j+b} \quad (6.3)$$

A 3×3 kernel has 9 parameters. We've gone from 10^{12} to 9—while encoding exactly the structure spatial data has.

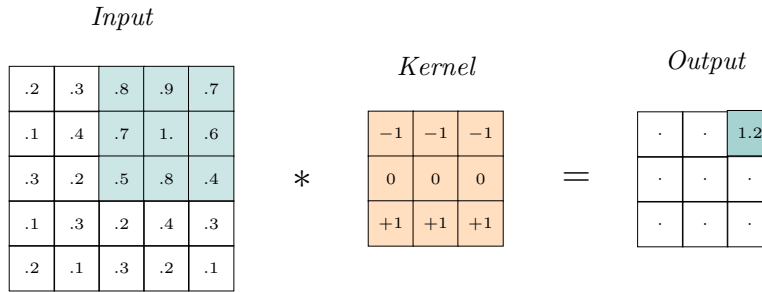


Figure 4. Convolution: a small kernel slides across the image, computing a weighted sum at each position. The same 9 weights are used everywhere (weight sharing). This kernel detects horizontal edges.

What convolutions learn. Each kernel extracts a local pattern. A horizontal edge detector:

$$\begin{pmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ +1 & +1 & +1 \end{pmatrix} \quad (6.4)$$

This computes the difference between rows below and above—large where there's a horizontal intensity change. In a CNN, kernels are *learned*. The network discovers which local patterns are useful for the task.

Channels. A single kernel detects one pattern. To detect multiple patterns, use multiple kernels in parallel. Each kernel produces one *output channel*—a feature map detecting that pattern across the image. A layer might have 64 or 256 channels, each detecting a different pattern.

The input can also have multiple channels: RGB images have 3 input channels; scientific images might have spectral bands or multiple observables. Each output channel combines information from all input channels:

$$h_{ij}^{(c)} = \sum_{c'} \sum_{a,b} W_{ab}^{(c,c')} x_{i+a,j+b}^{(c')} \quad (6.5)$$

The kernel $W^{(c,c')}$ maps input channel c' to output channel c . For a 3×3 kernel with 64 input and 128 output channels, this is $3 \times 3 \times 64 \times 128 \approx 74,000$ parameters—still far fewer than a fully connected layer.

Building hierarchies. Stack convolutional layers to build a hierarchy:

- **Layer 1:** Edges, gradients
- **Layer 2:** Corners, textures (combinations of edges)
- **Layer 3:** Shapes, parts (combinations of textures)
- **Layer 4:** Objects, concepts (combinations of parts)

Each layer’s *receptive field*—the input region affecting one output—grows with depth. After several layers, each output unit “sees” a large region.

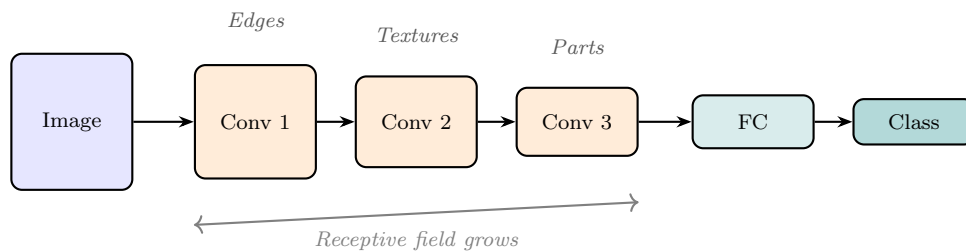


Figure 5. Hierarchical feature learning in CNNs. Early layers detect simple patterns (edges), later layers detect complex structures (objects). Receptive field grows with depth; spatial resolution decreases.

Pooling (taking the max or average over small windows) downsamples spatial dimensions, reducing computation and building robustness to small translations.

CNNs in science.— CNNs are the standard tool for any data on a grid: images (galaxy classification, medical imaging), spectra (stellar spectra, mass spectra), spatial fields (climate data, cosmological density fields). The same architecture that recognizes cats recognizes spiral galaxies—the inductive bias (translation invariance, locality) matches the data structure.

7 Sequential data: recurrent neural networks

Much scientific data is *sequential*: time series, spectra, trajectories, genetic sequences. The key property: **order matters**. The inductive bias: what happens now depends on what came before.

The problem. Given a sequence $x_1, x_2, \dots, x_T$, predict something—the next element, a classification, a property of the whole sequence. An MLP could flatten the sequence to a vector, but this ignores temporal structure. A CNN treats positions as spatial dimensions, which works but assumes fixed-length inputs and translation invariance in time (not always appropriate).

The idea. Process the sequence one element at a time, maintaining a *hidden state* h_t that summarizes what we’ve seen so far. Each new input updates the state:

$$h_t = f(h_{t-1}, x_t) \quad (7.1)$$

This is a **recurrent neural network** (RNN). The same function f is applied at every timestep—weight sharing across time, analogous to weight sharing across space in CNNs.

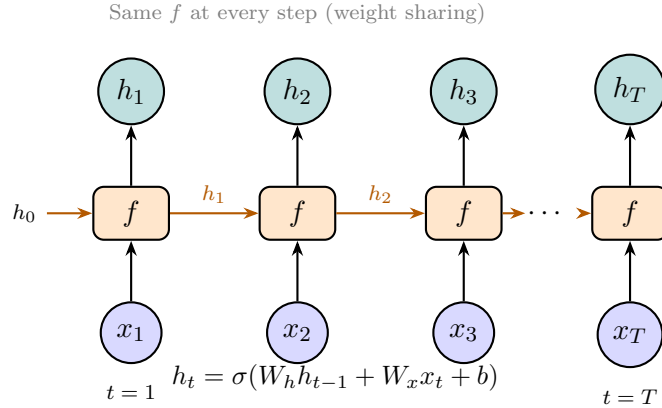


Figure 6. A recurrent neural network unrolled through time. The same function f (with shared weights) processes each input x_t and the previous hidden state h_{t-1} to produce the next hidden state h_t . The final state h_T summarizes the entire sequence.

The vanilla RNN. The simplest form:

$$h_t = \sigma(W_h h_{t-1} + W_x x_t + b) \quad (7.2)$$

Linear combination of previous state and current input, passed through a nonlinearity. This has the same structure as an MLP layer, but applied recurrently.

What can RNNs represent? RNNs are Turing-complete in principle—with enough hidden units, they can simulate any computation. In practice, they’re good at detecting patterns that depend on recent context: rhythms in time series, local motifs in sequences, short-range dependencies.

8 The trouble with vanilla RNNs

RNNs have a fundamental problem: **vanishing gradients**. To learn from long sequences, gradients must flow backward through many timesteps. But each step multiplies by W_h :

$$\frac{\partial h_T}{\partial h_1} = \prod_{t=2}^T \frac{\partial h_t}{\partial h_{t-1}} \approx W_h^{T-1} \quad (8.1)$$

If the largest eigenvalue of W_h is less than 1, gradients shrink exponentially. After 50 or 100 steps, the gradient from the end of the sequence barely reaches the beginning. The network can’t learn long-range dependencies—it forgets.

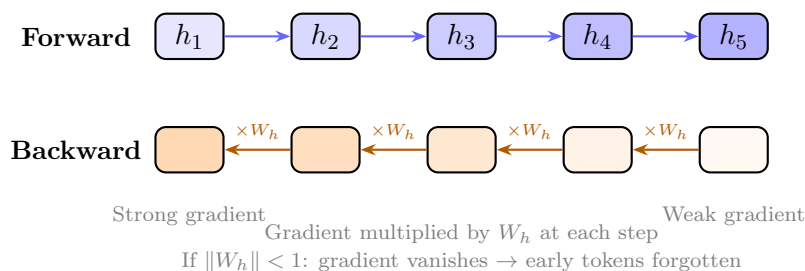


Figure 7. Vanishing gradients in RNNs. Gradients flow backward through time, multiplied by W_h at each step. If $\|W_h\| < 1$, gradients decay exponentially—early tokens contribute almost nothing to the loss gradient.

Gated architectures. The solution: add gates that control information flow. LSTMs (Long Short-Term Memory) and GRUs (Gated Recurrent Units) use learned gates to decide what to remember and what to forget. These provide a path for gradients to flow without repeated multiplication by W_h . They work well for sequences of a few hundred tokens but still struggle with very long sequences.

The fundamental limitation. Even with gating, RNNs process sequences *sequentially*—you can’t compute h_{100} without first computing h_1 through h_{99} . This makes training slow (no parallelization across time) and limits how far information can propagate.

Transformers, which we’ll discuss later, address this by processing all positions in parallel using attention. They’ve largely replaced RNNs for most sequence tasks, though RNNs remain useful when the sequential structure itself is important (e.g., modeling dynamical systems).

RNNs for scientific sequences.— RNNs are used for time series prediction in climate modeling, trajectory forecasting in molecular dynamics, sequence classification in genomics, and signal processing in detector data. When the temporal dynamics matter more than long-range dependencies, and when you need to process sequences of varying length online, RNNs are still a natural choice.

9 Beyond grids and sequences: relational data

CNNs assume data lives on a regular grid. RNNs assume data is ordered in time. But much scientific data has a different structure: entities connected by relationships.

- A **molecule** is atoms connected by bonds
- A **protein** is residues connected along a backbone
- A **physical system** is particles interacting pairwise
- A **point cloud** is unordered points in space

These are *graphs* (or *sets*, when there are no edges): nodes (entities) and edges (relationships). The structure is irregular—some atoms have many neighbors, others have few. There’s no grid, no canonical ordering.

The permutation problem. Consider a molecule A–B–C. We could label the atoms 1-2-3 or 3-2-1—both describe the same molecule. If we flatten the graph to a vector and feed it to an MLP, swapping labels changes the input and (potentially) the output. But the molecule hasn’t changed.

Permutation invariance: Functions on graphs must give the same output regardless of how we label the nodes. This is the inductive bias for relational data.

10 Deep Sets: the simplest permutation-invariant architecture

Before tackling graphs with edges, consider the simpler case: a *set* of elements with no relationships between them. A point cloud of particle positions. A bag of molecular fragments. A collection of measurements.

The question: how do we build a function $f(\{x_1, \dots, x_n\})$ that is invariant to the ordering of inputs?

The Deep Sets theorem. Any permutation-invariant function can be written as:

$$f(\{x_1, \dots, x_n\}) = \rho \left(\sum_{i=1}^n \phi(x_i) \right) \quad (10.1)$$

where ϕ processes each element independently and ρ processes the aggregated result.

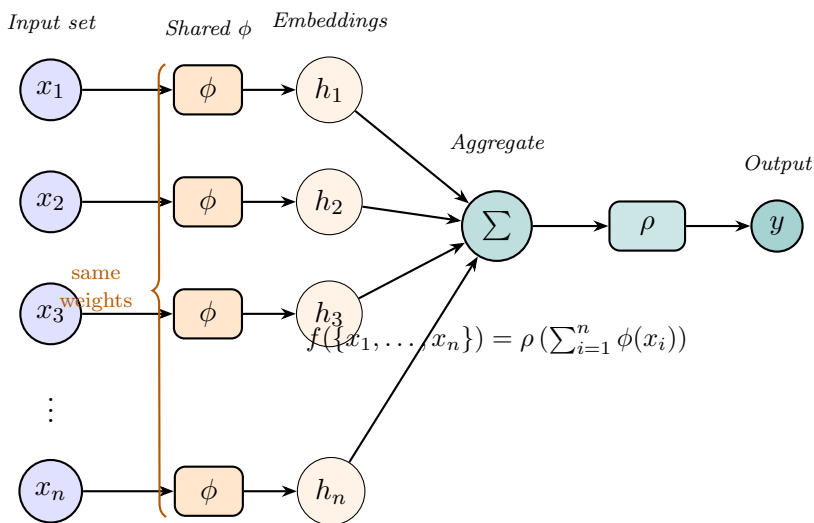


Figure 8. Deep Sets architecture. Each element x_i passes through the same network ϕ (weight sharing). Embeddings are summed (a symmetric operation), then processed by ρ to produce the output. Reordering the inputs doesn’t change the sum, so the output is permutation-invariant.

Why does this work? **Sum is permutation-invariant.** Reorder the inputs and the sum stays the same. The same holds for mean or max. This is the minimal architecture that respects the symmetry of sets.

Per-element outputs (equivariance). What if we want outputs for each element, not just the whole set? For example: given particle positions, predict a label for each particle. The architecture:

$$y_i = \psi \left(x_i, \sum_{j=1}^n \phi(x_j) \right) \quad (10.2)$$

Each output y_i depends on x_i and a global summary of the set. Permuting the inputs permutes the outputs in the same way—this is permutation *equivariance*.

Deep Sets is the foundation for all permutation-invariant architectures. GNNs extend it by adding edges.

11 GNNs: adding relational structure

A graph is a set with relationships. We have nodes $\mathcal{V}$ and edges $\mathcal{E}$ connecting them. The key operation in graph neural networks is **message passing**: nodes exchange information with their neighbors.

The intuition. Each node starts knowing only about itself. After one round of message passing, it knows about its immediate neighbors. After two rounds, neighbors-of-neighbors. After k rounds, each node has information about all nodes within k hops.

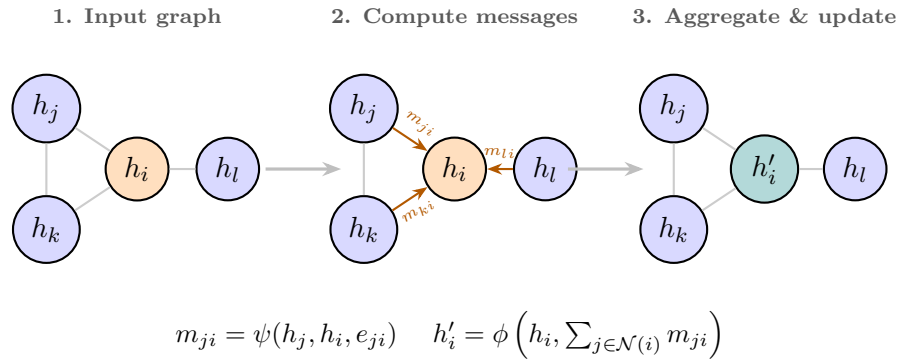


Figure 9. Message passing in three stages. (1) Start with node features on a graph. (2) Each node computes messages from its neighbors using a learned function ψ . (3) Messages are aggregated (summed) and combined with the node’s own features to produce updated representations.

The equation.

$$h_v^{(\ell+1)} = \phi \left(h_v^{(\ell)}, \sum_{u \in \mathcal{N}(v)} \psi(h_u^{(\ell)}, h_v^{(\ell)}, e_{uv}) \right) \quad (11.1)$$

Unpack this:

1. **Message:** For each neighbor u of node v , compute a message $\psi(h_u, h_v, e_{uv})$. The message depends on sender, receiver, and edge features.
2. **Aggregate:** Sum messages over all neighbors. This is where permutation invariance comes from—sum doesn’t depend on order.
3. **Update:** Combine aggregated messages with the node’s current state using ϕ to get the new representation.

Both ψ and ϕ are learned networks (typically MLPs). The same networks apply to all nodes and edges—*parameter sharing across the graph*.

Hidden dimensions (channels). Like CNNs, GNNs have a notion of channels: the hidden dimension d of the node representations $h_v \in \mathbb{R}^d$. A node’s “features” are a d -dimensional vector—you can think of each dimension as detecting a different pattern, just like channels in a CNN. Typical GNNs use 64–256 hidden dimensions. More dimensions mean more expressive power but also more parameters.

Connection to Deep Sets and CNNs. Message passing generalizes both:

- **Deep Sets** is a GNN on a graph with no edges (or a fully-connected graph with uniform edges).
- **CNN** is a GNN on a grid graph where each pixel connects to its spatial neighbors. The kernel weights are the message function.

From nodes to graphs. Message passing produces node representations h_v . For graph-level predictions (is this molecule toxic?), *pool* the nodes:

$$h_G = \sum_{v \in \mathcal{V}} h_v \quad (11.2)$$

This is exactly Deep Sets applied to the final node representations.

GNNs for molecules.— A molecule is a graph: atoms are nodes (features: element type, charge), bonds are edges (features: bond order). The task: predict properties (solubility, toxicity, binding affinity) from structure. GNNs are now standard for molecular property prediction, outperforming hand-crafted molecular fingerprints on most benchmarks.

12 Graph attention: learning which neighbors matter

Standard message passing treats all neighbors equally (or weights them only by edge features). But different neighbors may have different importance—and the importance may depend on the *content* of the nodes, not just the graph structure.

Attention learns to weight neighbors by relevance:

$$h'_v = \sum_{u \in \mathcal{N}(v)} \alpha_{uv} \cdot \psi(h_u) \quad (12.1)$$

The attention weights α_{uv} are computed from node features:

$$\alpha_{uv} = \frac{\exp(f(h_u, h_v))}{\sum_{w \in \mathcal{N}(v)} \exp(f(h_w, h_v))} \quad (12.2)$$

The scoring function f (typically a small network or inner product) computes how relevant neighbor u is for node v . The softmax ensures weights sum to 1.

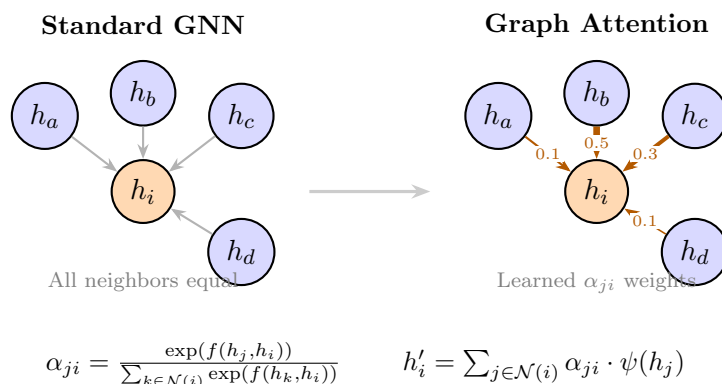


Figure 10. Graph attention learns which neighbors matter. Left: standard GNN weights all neighbors equally. Right: attention weights neighbors by learned relevance scores α_{ji} , allowing the network to focus on the most informative connections.

Why attention helps. Consider predicting a molecule’s reactivity. A carbon atom may have several neighbors, but not all matter equally—the oxygen in a carbonyl group matters more than a distant methyl. Attention learns these patterns from data.

Attention also provides a form of interpretability: the learned weights α_{uv} show which neighbors influenced each node’s representation.

From graphs to sequences. The attention mechanism here operates on fixed graph structure—neighbors are determined by edges. *Transformers* generalize this: every element attends to every other element, with attention weights determining connectivity rather than a predefined graph. Flexible, but computationally expensive (quadratic in sequence length). We’ll return to transformers when we discuss language models.

13 Expressivity: what can networks learn?

We’ve introduced several architectures. A natural question: what functions can each one represent?

Universal approximation. MLPs can approximate any continuous function, given enough neurons. CNNs with enough channels can approximate any translation-equivariant function. GNNs can approximate any permutation-equivariant function (with caveats). These are *expressivity* results: they say what’s possible in principle.

But expressivity isn't everything. A universal approximator with 10^{100} parameters is useless. What matters is:

1. **How many parameters** are needed to approximate a given function class?
2. **How much data** is needed to learn those parameters?
3. **Can optimization find** the right parameters?

Inductive biases help with all three. A CNN needs far fewer parameters than an MLP to learn translation-invariant functions. A GNN needs less data to learn permutation-invariant functions than an MLP trained with data augmentation. And the constrained parameter space is often easier to optimize.

The bias-variance tradeoff. More structure means less flexibility. A CNN *cannot* learn position-dependent functions (e.g., “objects in the top-left are more important”). A GNN *cannot* learn functions that depend on node ordering. If these restrictions match your data, they help. If not, they hurt.

GNN expressivity. Standard message-passing GNNs have a known limitation: they cannot distinguish certain graph structures that the Weisfeiler-Lehman graph isomorphism test cannot distinguish. Intuitively, if two graphs “look the same” from the perspective of local neighborhoods, a GNN will compute the same output. More expressive architectures (higher-order GNNs, subgraph GNNs) can distinguish more graphs, at higher computational cost.

14 The design principle

We've seen a progression of architectures:

Data type	Architecture	Inductive bias
Vectors	MLP	None (maximum flexibility)
Images/grids	CNN	Translation invariance + locality
Sequences	RNN	Temporal order, weight sharing
Sets	Deep Sets	Permutation invariance
Graphs	GNN	Permutation invariance + edges

The pattern: **match architecture to data structure**. If you know the data has symmetries, encode them. This reduces the hypothesis space, improving sample efficiency and generalization.

But don't over-encode. If your assumptions are wrong—if images *do* have position-dependent structure, if sequences *don't* have temporal order—the inductive bias hurts rather than helps.

Physical systems have additional symmetries we haven't yet encoded: rotation and translation in 3D space. A molecule's energy doesn't change if you rotate it. We'll develop architectures that respect these continuous symmetries in the next chapter.

Key Takeaways

- Neural networks compose simple nonlinear functions to learn complex relationships. **Nonlinearity** enables learning complex decision boundaries.
- **Residual connections** enable training very deep networks by providing direct gradient paths.
- **CNNs** encode translation invariance and locality through weight sharing (convolution). The inductive bias: patterns are local and position-independent.
- **RNNs** process sequences by maintaining hidden state across time. They struggle with long-range dependencies due to vanishing gradients. The inductive bias: the present depends on the past.
- **Deep Sets** is the minimal permutation-invariant architecture: process elements independently, aggregate with sum/mean/max.
- **GNNs** extend Deep Sets with relational structure via message passing. The inductive bias: nodes should exchange information with their neighbors.
- **Graph attention** learns which neighbors are most relevant. Transformers generalize this to sequences.
- **Expressivity** tells us what networks can represent; **inductive bias** determines how efficiently they can learn it from data.